CSCI1409 Deep Learning · Week 5

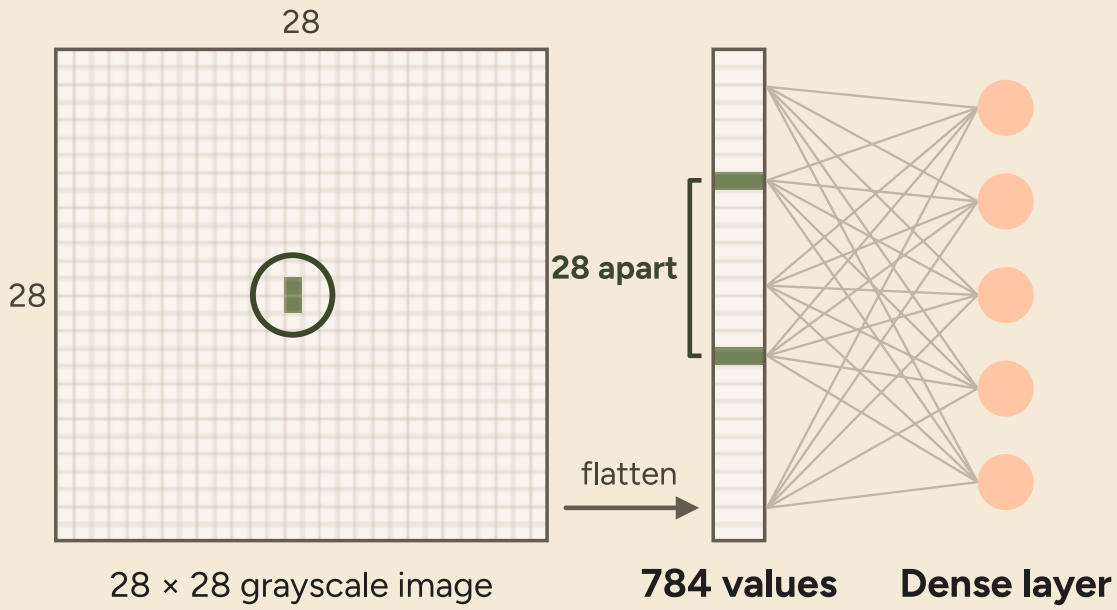
Convolutional Neural Networks: Foundations

Aidyn Abirov · KBTU

Lecture 12:00–13:00 · Fashion-MNIST SIS 13:00–14:00

Image structure and dense layers

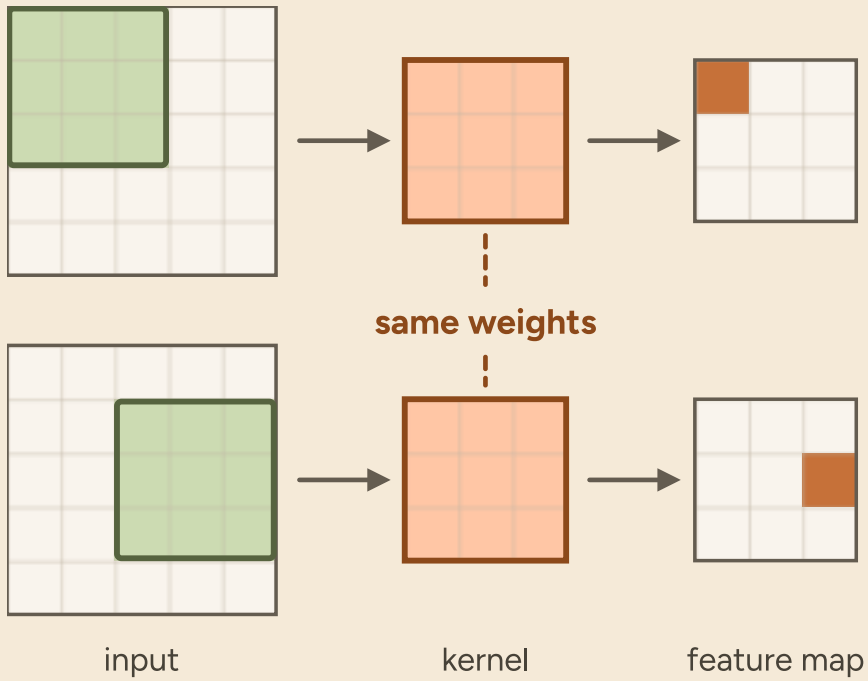
- Fashion-MNIST: 28 × 28 grayscale images, 10 classes
- One image: 1 × 28 × 28 → 784 pixel values
- Dense layers can classify images
- CNNs build in local connections and shared weights



Data: Fashion-MNIST · github.com/zalandoresearch/fashion-mnist

A shared kernel over local patches

- A kernel looks at a local patch
- Multiply matching entries, sum, add a bias
- Reuse the same weights at each position
- The outputs form a feature map



Source: PyTorch 2.14 docs · torch.nn.Conv2d

One convolution by hand

Kernel 3 × 3 · stride 1 · padding 0 · bias 0

synthetic input

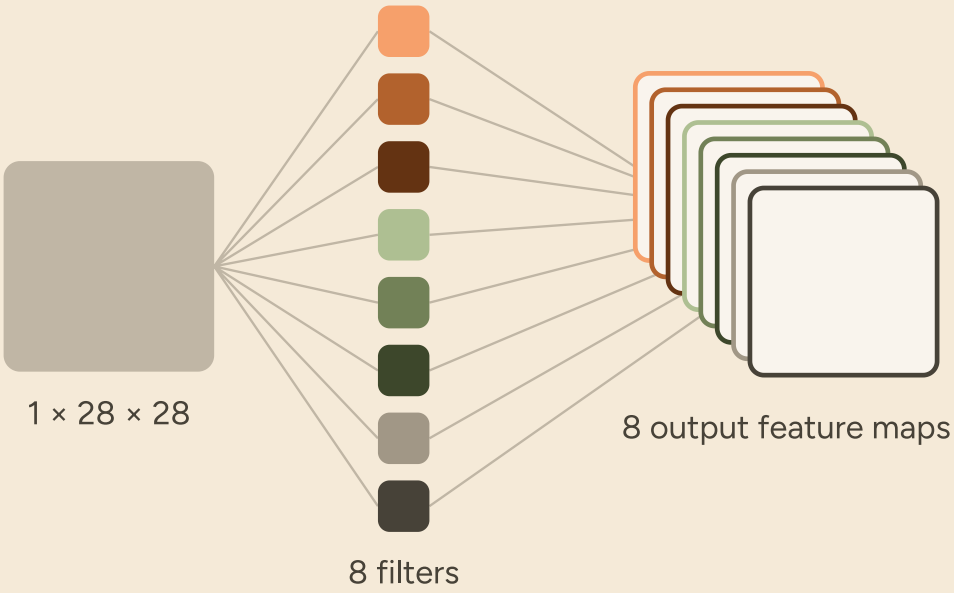
$$X = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 3 & 2 \\ 2 & 2 & 1 & 0 \\ 1 & 0 & 2 & 3 \end{bmatrix} \qquad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \qquad Y = \begin{bmatrix} -1 & \square \\ \square & \square \end{bmatrix}$$

First output: $(1 - 0) + (0 - 3) + (2 - 1) = -1$

What is the top-right output?

Channels and learned parameters

- Each output channel has its own learned filter
- Each filter spans all input channels
- Parameters $= C_{\text{out}}(C_{\text{in}}\, k_{\text{h}}\, k_{\text{w}} + 1)$
- Example: $8(1 \times 3 \times 3 + 1) = 80$



Layer 2 weights

$$16 \times 8 \times 3 \times 3$$

output channels · input channels · kernel height · kernel width

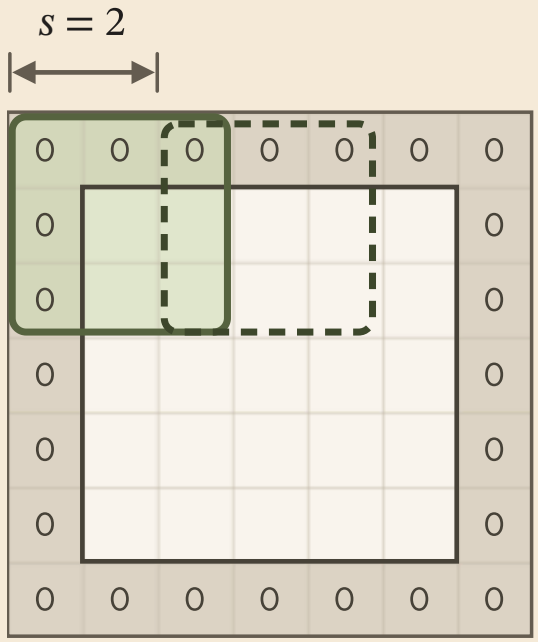
Source: PyTorch 2.14 docs · torch.nn.Conv2d

Stride, padding, and output size

- Stride s: distance between kernel positions
- Padding p: added border values
- For dilation 1:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p_{\text{h}} - k_{\text{h}}}{s_{\text{h}}} \right\rfloor + 1$$

$$W_{\text{out}} = \left\lfloor \frac{W + 2p_{\text{w}} - k_{\text{w}}}{s_{\text{w}}} \right\rfloor + 1$$



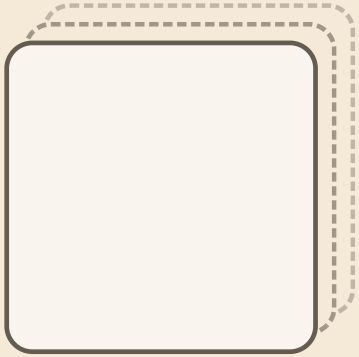
$p = 1$ zero border

Source: PyTorch 2.14 docs · torch.nn.Conv2d

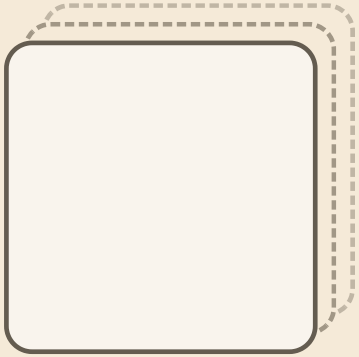
Output-shape practice

Input: B × 1 × 28 × 28 · 8 filters · kernel 3 × 3

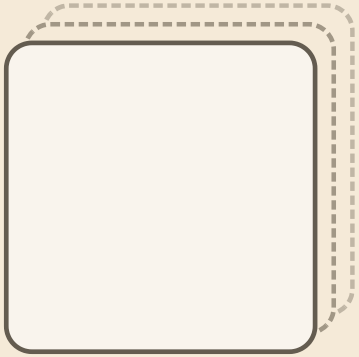
A: p = 0, s = 1 → ?



B: p = 1, s = 1 → ?



C: p = 1, s = 2 → ?



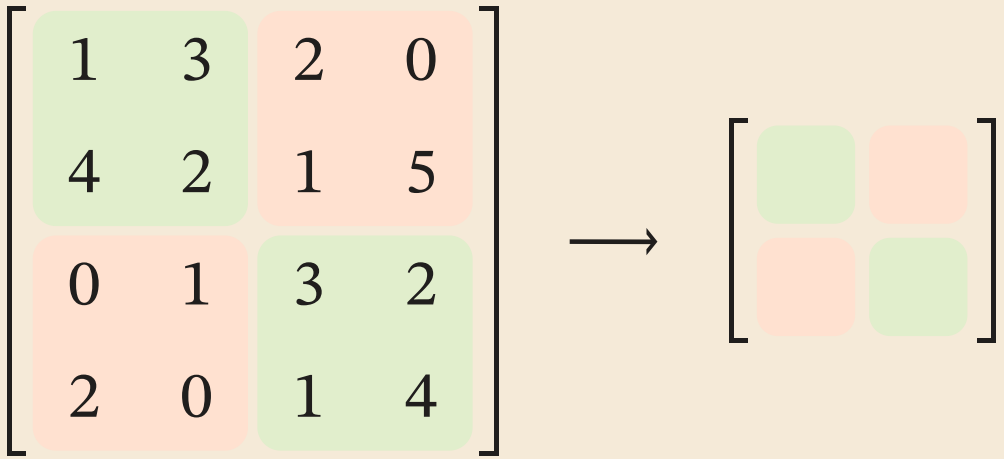
ReLU on a feature map

- $\mathrm{ReLU}(z) = \max(0, z)$
- Apply it to each entry
- Same shape · no learned parameters

$$\begin{bmatrix} -1 & 2 \\ -3 & -2 \end{bmatrix} \longrightarrow \begin{bmatrix} 0 & 2 \\ 0 & 0 \end{bmatrix}$$

Max pooling

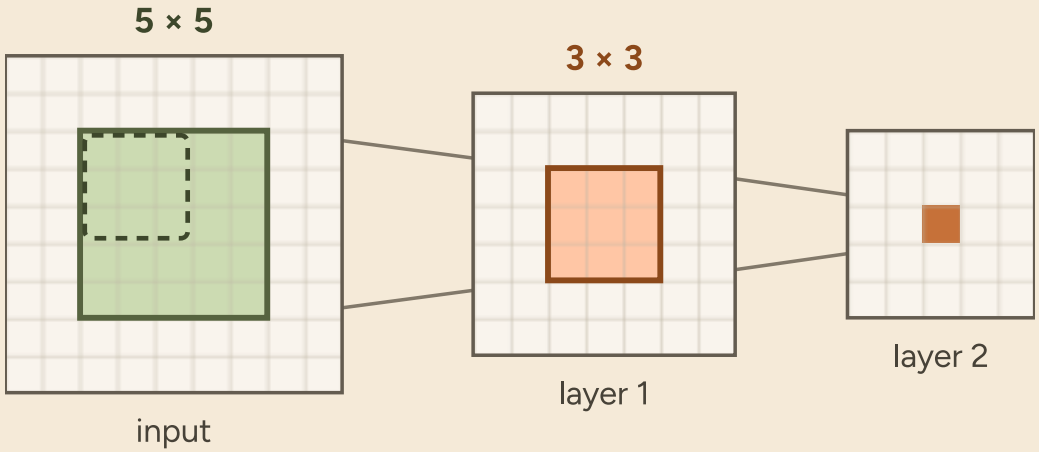
- 2 × 2 windows · stride 2 · no padding
- Keep the maximum in each window
- Process each channel independently
- Fewer spatial positions · no learned parameters



B × 8 × 28 × 28 → B × 8 × 14 × 14

Depth and receptive fields

- Receptive field: input region that can affect an activation
- One 3 × 3 convolution → 3 × 3 input region
- Two 3 × 3 convolutions → 5 × 5 input region
- Here: stride 1, no pooling between layers



interior activation; schematic

A small Fashion-MNIST CNN

Conv → ReLU → Pool → Conv → ReLU → Pool

Flatten → Dense classifier → 10 logits



What is the flattened feature count?

Data: Fashion-MNIST (Zalando Research) · Layers: PyTorch 2.14 nn.Conv2d docs

The CNN in PyTorch

```
model = nn.Sequential(
    nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2), nn.Flatten(),
    nn.Linear(16 * 7 * 7, 10)
)
logits = model(x)  # [B, 10]
loss = nn.CrossEntropyLoss()(logits, y)
```

x: [B, 1, 28, 28] · y: [B], integer labels 0–9

Sources: PyTorch 2.14 docs · nn.CrossEntropyLoss · Optimization tutorial

Recap and Fashion-MNIST SIS

1. Why are the kernel weights shared?
2. What shape reaches Flatten in today's CNN?
3. What stays the same in the training process?

13:00–14:00: Fashion-MNIST SIS